Technical Assessment - Bug fixes and Improvements

1] SQL filtering Bug.

bug 1 :- Incorrect task filtering.

* issue :- the Task search query had an incorrect combination of AND and OR conditions.

* How I found it :
I reviewed the SQL query in TaskRepository and check how SQL evaluates AND and OR.

* what I changed :-
= archieved = FALSE
AND (
title matches search
OR description matches search
)
AND status matches filter.

* why :- This ensures archieved task are excluded and both search and status filter works correctly.

2] Pagination + status.

* Issue :- when I was on a later page and changed the search or status filter, the page number was not reset.

* How I found it :-
I checked App.jsx and noticed that changing query and status did not change the page state

* what I changed :-

handleQueryChange()
setQuery(value)
setPage(1)
handleStatusChange()
setStatus(value)
setPage(1)

* why :- A new search or filter should start from page 1.

Bug 3:- Invalid status.

* Issue:- The backened ~~issue~~ used ~~Task~~ TaskStatus.valueOf() without handling invalid values.

* How I found it:- I reviewed TaskController.java.

An invalid value such as:
status = INVALID
could throw IllegalArgumentException

* what I changed:
I added try-catch around TaskStatus.valueOf().

Invalid status now returns
HTTP 400 Bad request
with an error message.

* Why:- Invalid user input should be handled safely instead of causing an unexpected ~~server~~ error.

Bug 4] Frontend + Performance

* Issue:- The frontend did not correctly reset the loading state when an API request failed

The ~~pre~~ previous error could also remain when a new request started.

* How I found it:- I reviewed useTask.js.

* what I changed:-
setLoading(true)
setError(null)
success → update tasks
error → set error and clear tasks.
finally → setLoading(false)

* Why:- this makes the UI correctly show loading, success and error states.

Bug 5] Artifical API Delay.

* Issue:- The backend contained Thread.sleep() which deliberately delayed every API request.

* How I found it:
  While reviewing TaskController.java, I found a query complexity calculation followed by Thread.sleep().

* what I changed:-
  I remove artifical delay and unnecessary complexity calculations.
  I kept the request logging.

* why:-
  Thread.sleep() block the request thread and makes the application unnecessarily slow.

* Result:- Task searches respond without the artifical delay.

Bug 6] Invalid Pagination & Parameters.

* Issue:- The API accepted invalid values such as:
  page = 0
  page = -1
  pageSize = 0.

* How I found it
  I checked the pagination calculations.
  start = (page - 1) * pageSize.
  There was no validation before this calculations.

* what I changed:
  I added validation:
  page must be >= 1
  pageSize must be > 0
  Invalid values now return:
  HTTP 400 Bad request.

* why: This prevents incorrect pagination calculation and unexpected behaviour.

Bug 7] Stale API Responses.

Issue :- Multiple search request can be sent when the user changes the search quickly. An older request could finish after a newer request and overwrite the latest results.

How I found it :- I reviewed the asynchronous API handling in useTasks.js.

What I changed :-
I added a cleanup mechanism using a cancelled flag.

Request starts
↓
User changes search
↓
Previous request is cancelled
↓
Only latest request update UI.

Why :- This prevent stale search result from replacing newer results.

Bug 8] Task ordering.

Issue :- The Task list was displaying in descending order.

How I found it :-

I checked the order by clause in TaskRepository.java.

What I change :-
ORDER BY id ASC;

Why :- Asc Ascending order makes the task list and pagination easier to follow.